

RESUMEN TPG — React Ecommerce

Context API y useContext

Compartir datos entre componentes sin prop-drilling. `createContext()` → `Provider` → `useContext()`. Tres contextos en el proyecto:

FavoriteContext → `favoriteItems`, `addToFavorite`, `toggleFavorite`, `isFavorite`

CartContext → `cartItems`, `addToCart`, `removeFromCart`, `total`, `itemCount`

AuthContext → `isAuthenticated`, `user`, `login`, `register`, `logout`

```
const ctx = createContext();

export const useCart = () => useContext(ctx);

// En App.jsx:
```

FavoriteProvider

Mantiene `favoriteItems` en `useState`. `toggleFavorite` agrega o quita; `isFavorite(id)` retorna `true/false`.

```
const toggleFavorite = (product) => {
  isFavorite(product.id)
    ? removeFromFavorite(product.id)
    : addToFavorite(product);
};
```

Heart button en ProductCard

`useFavorite()` → `toggleFavorite` + `isFavorite`. `e.preventDefault()` evita navegar al detalle. Ternario para el ícono:

```
{isFavorite(product.id) ? "♥" : "■"}

onClick={(e) => { e.preventDefault(); toggleFavorite(product); }}
```

useNavigate

Navegar programáticamente después de acciones (`login`, `compra`). A diferencia de `<Link>`, se llama dentro de funciones.

```
const navigate = useNavigate();

navigate("/"); // home

navigate(-1); // atrás
```

useParams

Lee parámetros dinámicos de la URL. En ruta `/products/:id` devuelve `{ id: "5" }`. Se usa en `ProductDetail` para el `fetch`.

```
const { id } = useParams();

fetch(`http://localhost:8080/api/productos/${id}`)
```

Rutas Protegidas

`ProtectedRoute` envuelve rutas privadas. Si `!isAuthenticated` redirige a `/login` con `<Navigate>`. `/checkout` y `/favorites` están protegidas.

```
const { isAuthenticated, loading } = useAuth();

if (loading) return <Spinner/>;

return isAuthenticated ? children : <Navigate to="/login"/>;
```

Renderizado Condicional

if/else fuera del JSX → para carga y errores. **&&** en JSX → muestra solo si `true`. **Ternario ?:** → elige entre dos opciones.

```
{loading && <Spinner/>}

{vacío ? <Mensaje/> : <Lista/>}

{itemCount > 0 && <Badge>{itemCount}</Badge>}
```

Operadores Ternarios — Ejemplos del proyecto

```
pathname === path ? "navbar__link--active" : ""

isFavorite(id) ? "♥" : "■"

stock > 0 ? "Disponibile" : "Sin stock"

loading ? "Cargando..." : "Ingresar"
```

.map()

Transforma cada elemento del array en JSX. Requiere prop `key` única (usar `id`, nunca índice).

```
{favoriteItems.map(product => (
```

```
<ProductCard key={product.id} product={product} />  
  )}]
```

[Context API](#) · [useNavigate](#) · [useParams](#) · [Rutas Protegidas](#) · [Renderizado Condicional](#) · [.map\(\)](#)